Parallelism Basics

DSC 102: Systems for Scalable Analytics

Haojian Jin

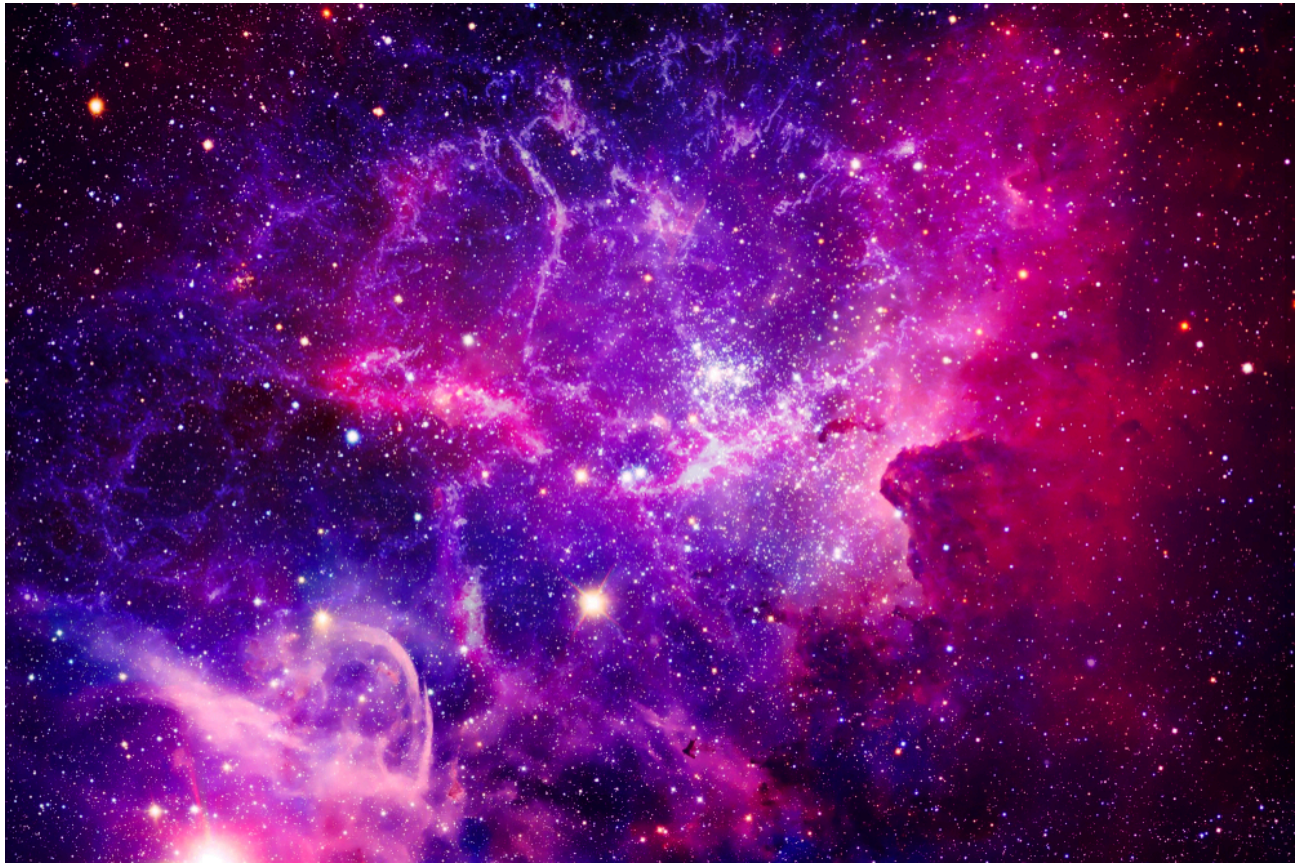
UC San Diego

Where we are

- **L1:** the hardware: processors, caches, DRAM, disks
- **L2:** the agent above the API, and its memory
- **L3:** the operating system sharing one machine among many processes
- **Today:** what to do when **one processor is not enough**

Parallelism basics: task graphs, task parallelism, Dask, and the parallel hardware inside one node.

Q: Why bother with large-scale data?



Why does **sampling** not suffice?

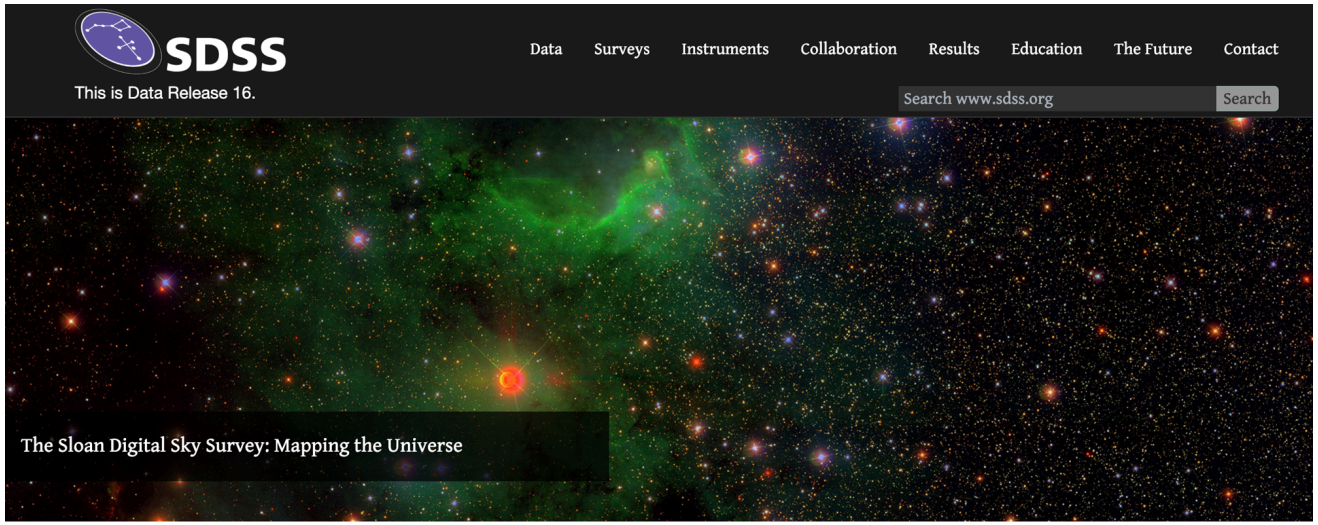
If a 1% sample fits in memory and gives the right mean, why pay for the other 99%?

Think about it for a minute; we will see four answers.

Five questions for today

1. Why **large-scale data**?
2. How do we **describe** a parallel computation?
3. How does **task parallelism** work, and how much does it help?
4. How does **Dask** do it?
5. What parallelism lives **inside one node**?

Q1 · Astronomy

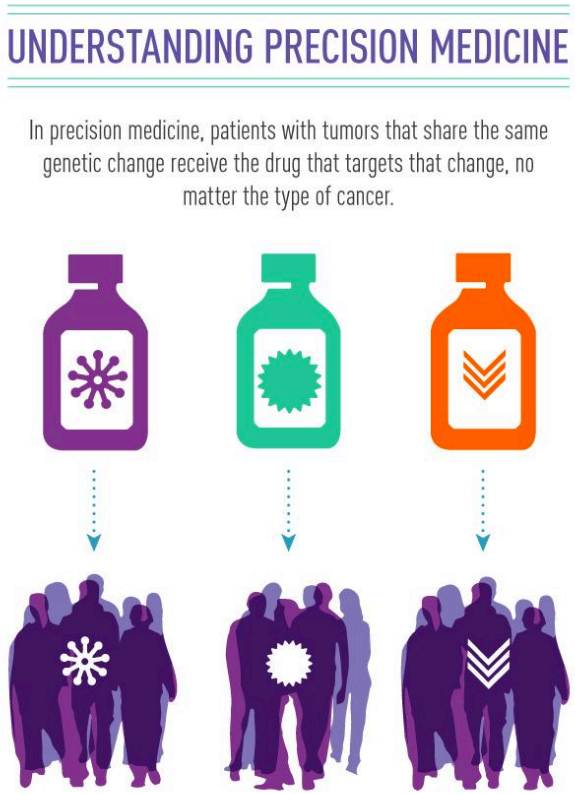


The **Sloan Digital Sky Survey**:

- High-resolution images, **~200 GB per day** since 2000: over a petabyte
- Astronomers can study complex galactic evolution behaviors
- Rare objects are rare: you cannot sample your way to them

sdss.org

Genomics



Precision medicine is becoming a reality.

- Analyze genomes across cohorts and prescribe targeted drugs and treatments
- **~3 GB** per human genome
- **~1 EB** for the USA

Signal lives in rare variants across many people.

E-commerce



Netflix logs **all** user behavior: views, clicks, pauses, searches.

- Recommender systems combine **terabytes** of data from all users and all titles to tailor one page
- Over 80% of what people watch comes from recommendations

The long tail of users and titles is where personalization lives.

Netflix Tech Blog, "Everything is a recommendation".

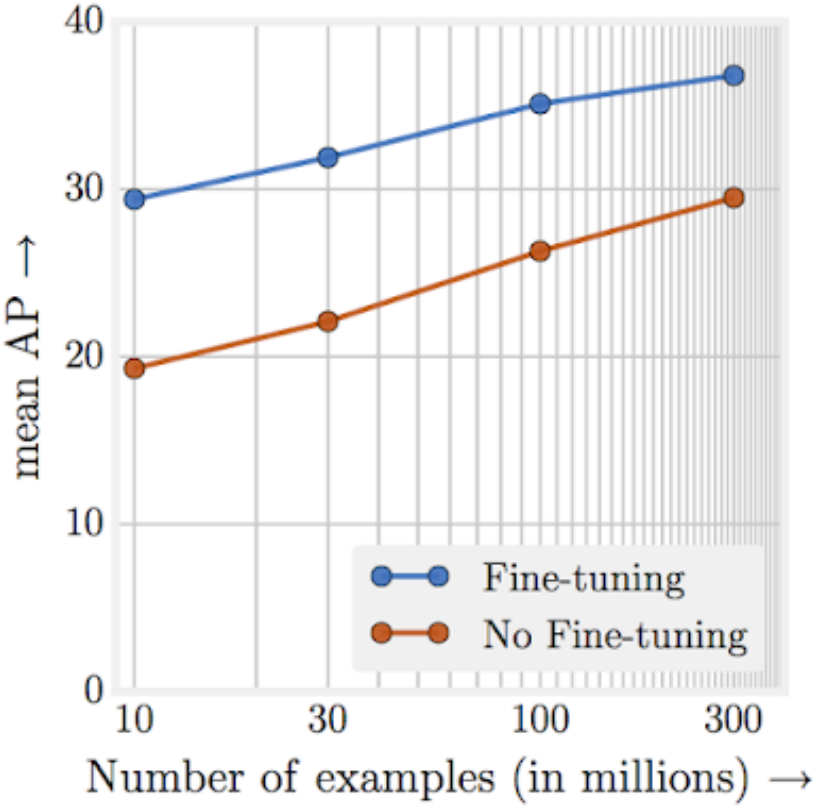
Computer vision



ImageNet:

- 10 million+ images labeled by crowdsourcing into 20,000 classes
- Over **500 GB** uncompressed as tensors
- The harbinger of the deep learning revolution (2012)

"The unreasonable effectiveness of data"

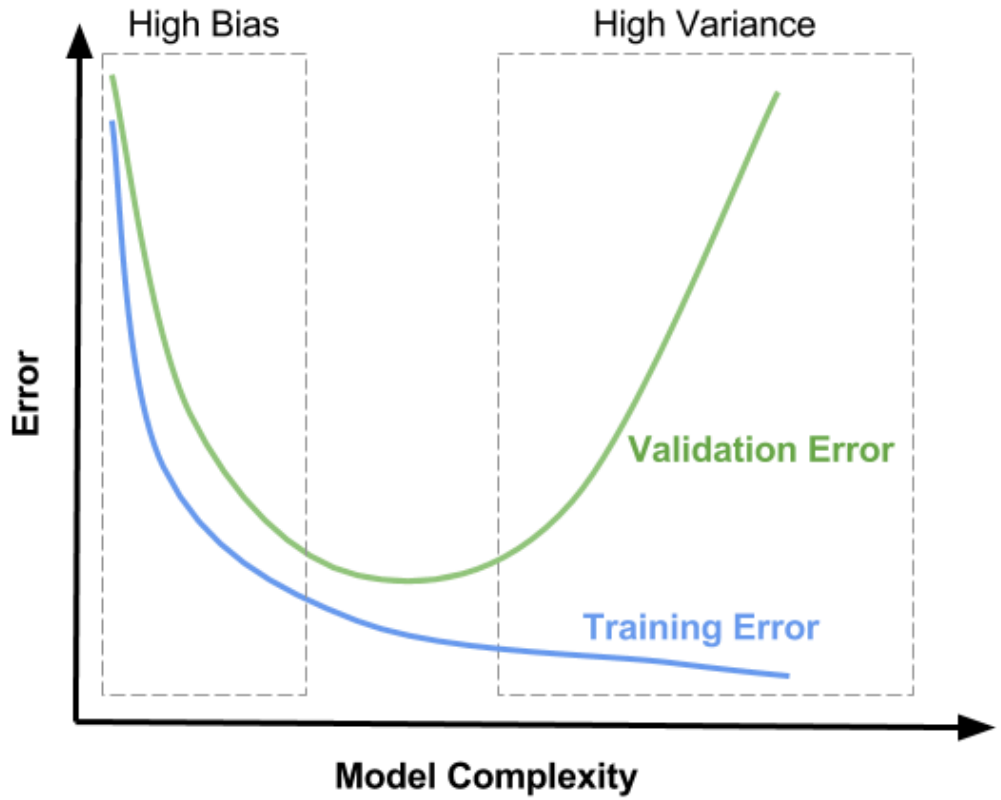


When the prediction target is complex, **more training data** coupled with **more complex models** keeps raising accuracy as the number of examples grows.

Here: object detection mean AP keeps climbing from 10M to 300M training images.

Sun et al., Revisiting Unreasonable Effectiveness of Data in Deep Learning Era, ICCV 2017. https://ai.googleblog.com/2017/07/revisiting-unreasonable-effectiveness.html

The bias–variance tradeoff



- **High bias:** the model is not rich enough to represent the data
- **High variance:** the model overfits the given data and generalizes poorly

Large-scale training data lowers variance, so richer models become safe to use, and accuracy rises.

Why large-scale data?

Large-scale data is a game changer in data science:

- Enables the study of **granular phenomena** in the sciences and in business that was not possible before
- Enables **new applications**, personalization, and customization
- Enables more complex ML prediction targets and **mitigates variance** for higher accuracy

And the hardware kept pace:

- Storage capacity exploded (petabyte clusters); compute grew (multi-core, GPUs); DRAM grew from tens of GB to TB
- Public cloud IaaS democratized access to hardware; open source and SaaS democratized software

"Big Data"

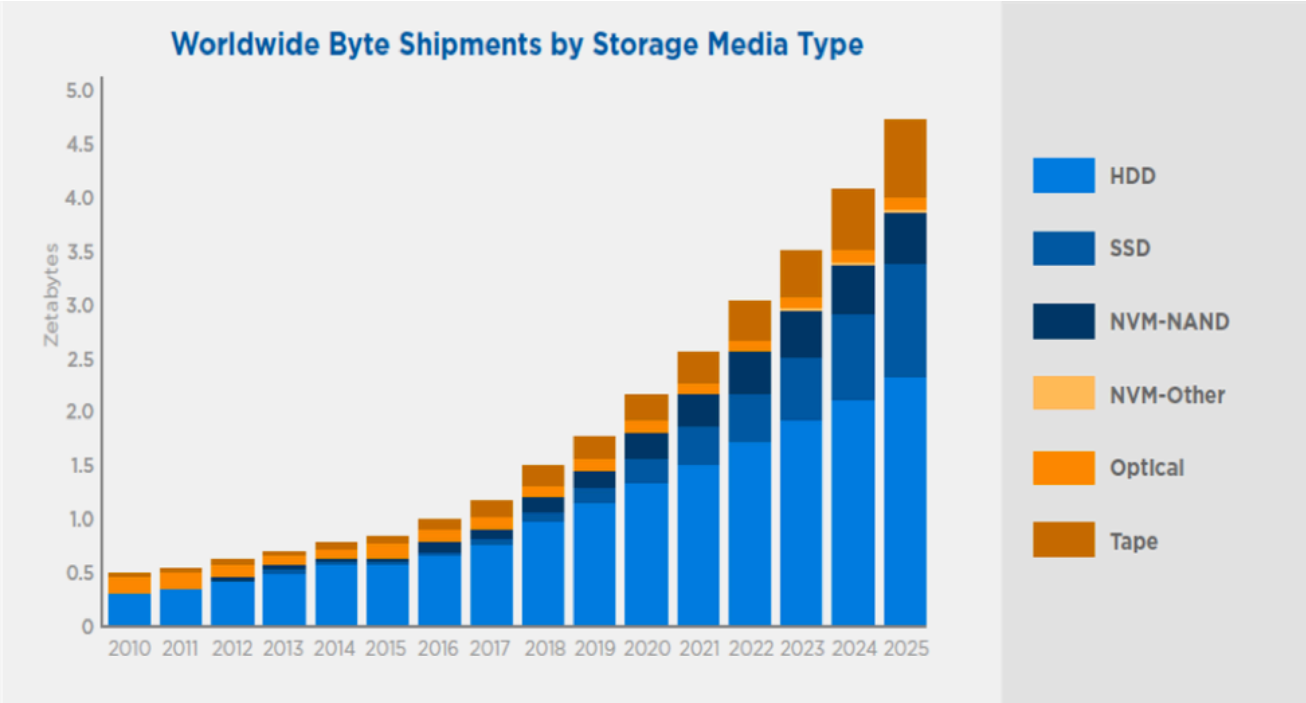
A **marketing term**: think "Big" as in Big Oil or Big Tech, not "big building". Popular from the late 2000s.

Wikipedia: *"data that is so large and complex that existing toolkits [read: RDBMSs] are not adequate."*

The usual characterization, the **three Vs**:

Volume	larger than a single node's DRAM
Variety	relations, documents, tweets, multimedia, …
Velocity	high generation rate: sensors, surveillance, logs

Why "Big Data" now?



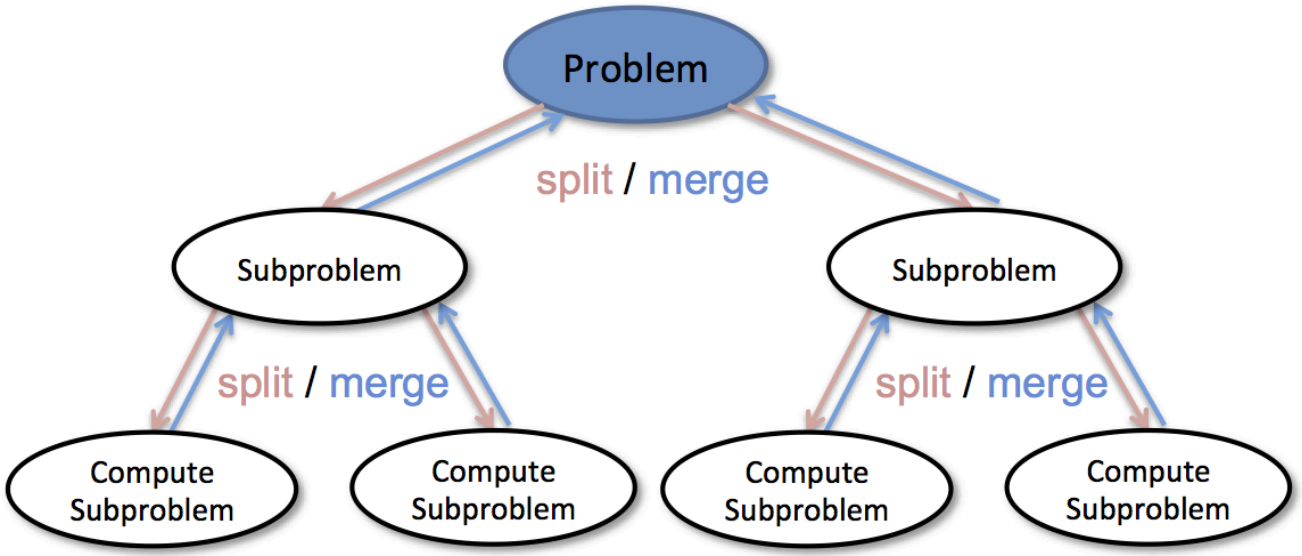
1. Applications: a data-driven mentality in almost every human endeavor: web search and social media, satellite imagery and the LHC, precision medicine, IoT sensors, high-frequency trading, digitized literature, e-voting.

2. Storage: worldwide byte shipments keep climbing, and most of it lands on HDD and SSD.

Data Age 2025, IDC/Seagate (2018).

To analyze large-scale data, parallel and scalable data systems are indispensable.

Q2 · Parallel data processing



Central issue: the workload takes too long for one processor.

Basic idea: split the workload across processors, and perhaps across machines ("workers"): **divide and conquer**.

The question is *what* to split: the tasks, or the data.

Reminder: threads

- A **thread** generalizes the OS process abstraction: a multithreaded process spawns many threads, each running part of the program's computation at the same time
- All threads **share the address space**, so they share the data too
- In a multi-core CPU, a running thread uses up one core; **hyper-threading** virtualizes one core to run two threads
- Common in parallel data processing: Dask workers can be threads, processes, or both

One process, several threads, several cores: parallelism without copying the data.

A new concept: the dataflow graph

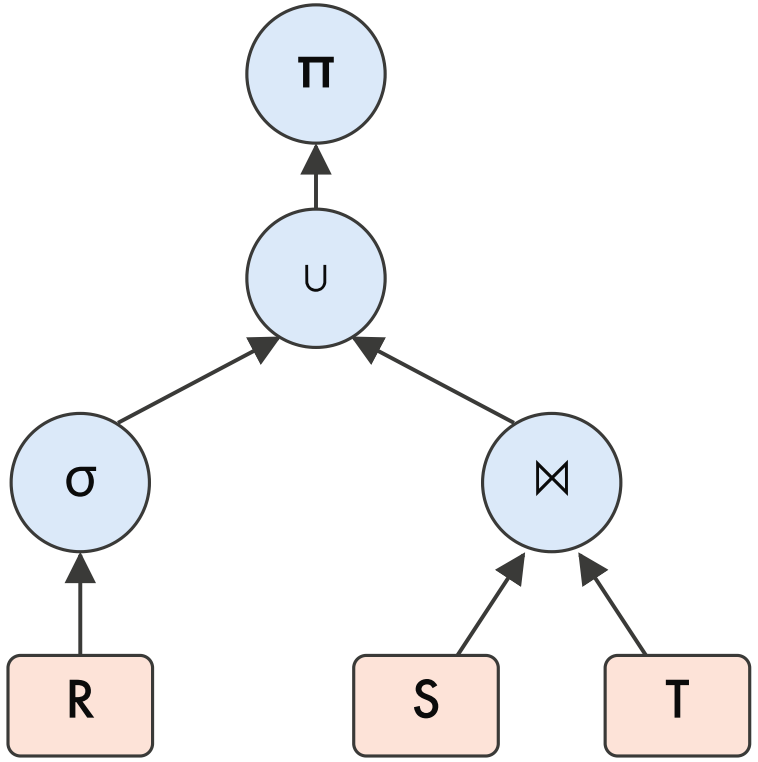
Dataflow graph: a directed graph representation of a program whose vertices are **abstract operations** from a restricted set of computational primitives.

- **Extended relational dataflows:** RDBMSs, pandas, Modin
- **Matrix and tensor dataflows:** NumPy, PyTorch, TensorFlow

It lets us reason about data-intensive programs at a **higher level**, the logical level.

Task graph: similar, but coarse-grained: a vertex is a whole process.

Example: a relational dataflow graph



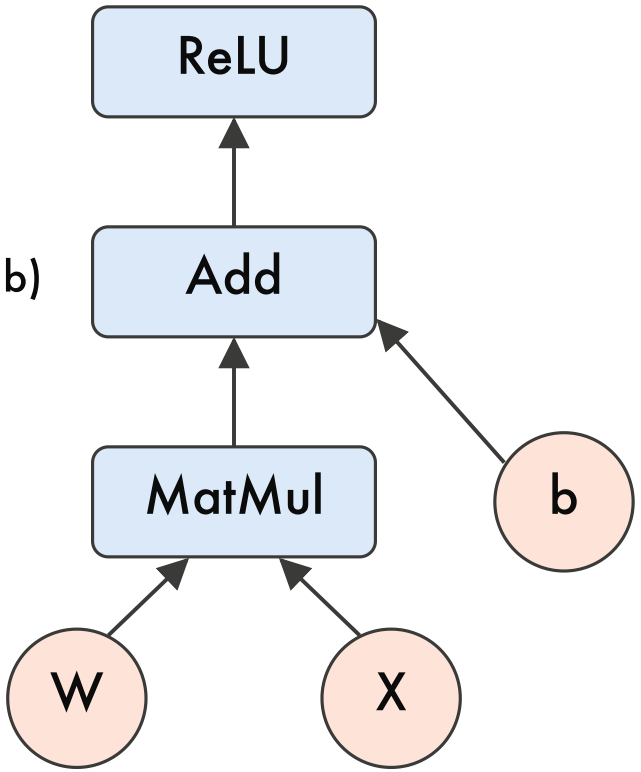
$\pi(\sigma(R) \cup (S \bowtie T))$

- Vertices are **operators** from the extended relational algebra: select σ, project π, join ⋈, union ∪
- Edges carry **intermediate data**
- Leaves are the **input relations** R, S, T

Database people call this a **logical query plan**.

Example: a tensor dataflow graph

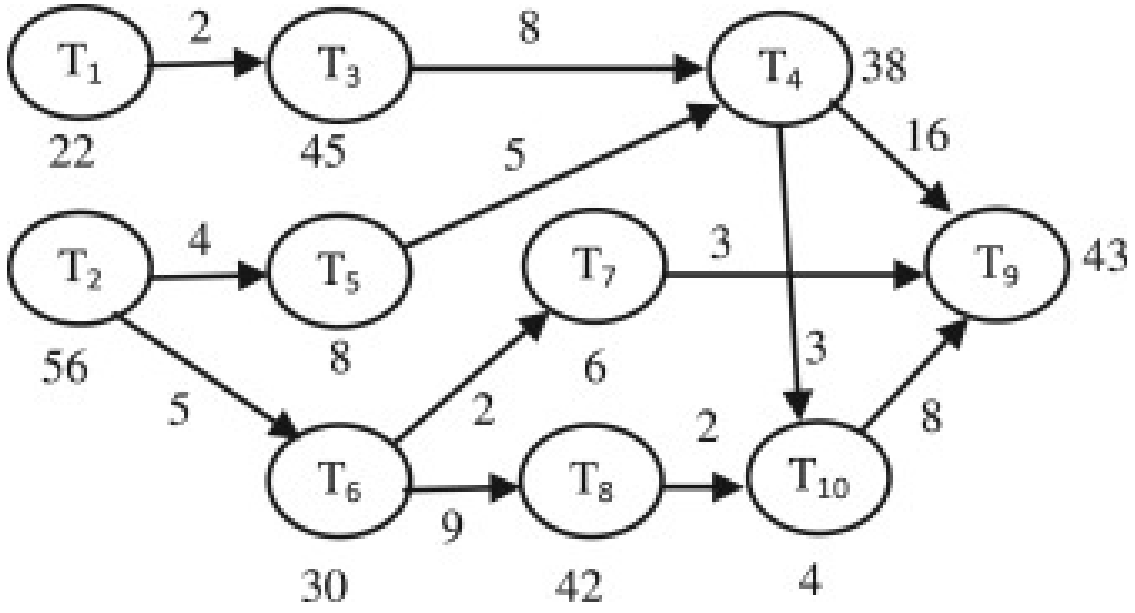
h = ReLU(WX + b)



- Vertices are **operators** from the tool's tensor algebra: MatMul, Add, ReLU
- Edges carry **intermediate tensors**
- Leaves are the **inputs**: weights W, bias b, batch X

ML systems people call this a **neural computational graph**.

Example: a task graph

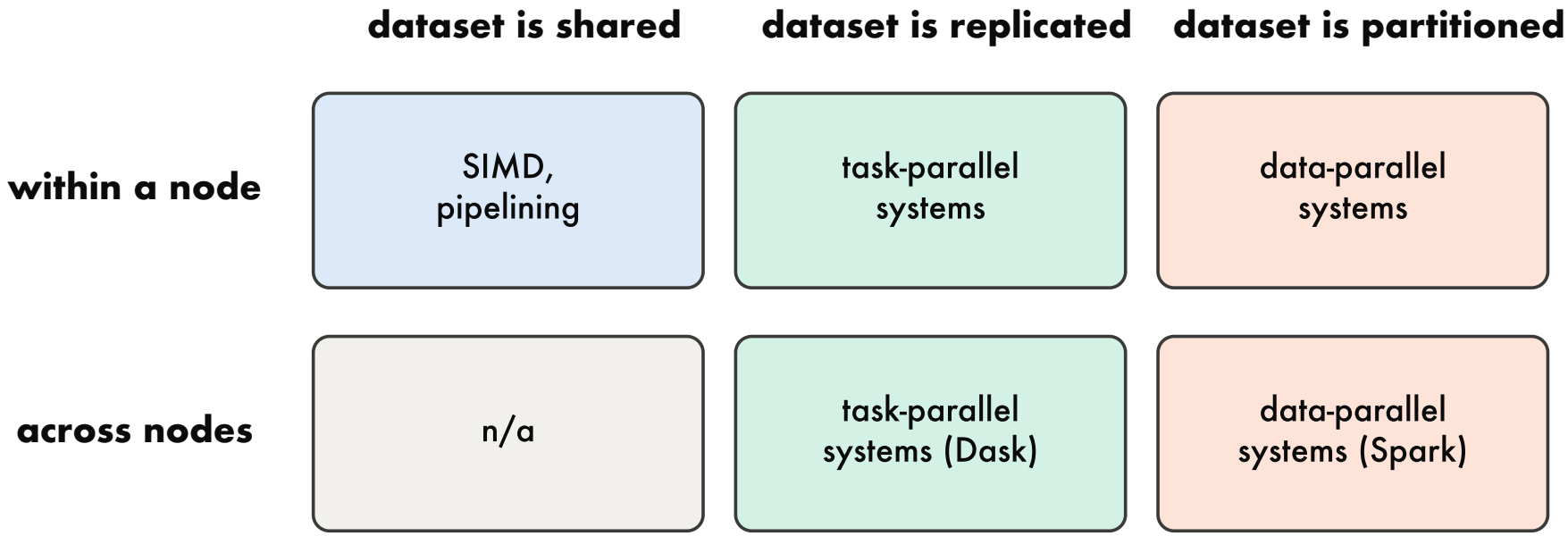


- More coarse-grained than operator-level dataflows
- **Vertex:** a full task or process; **edge:** a dependency between tasks
- Usually a **directed acyclic graph (DAG)**; the data may not be shown

Dask conflates the two: an operation on a Dask DataFrame becomes its own separate task under the hood.

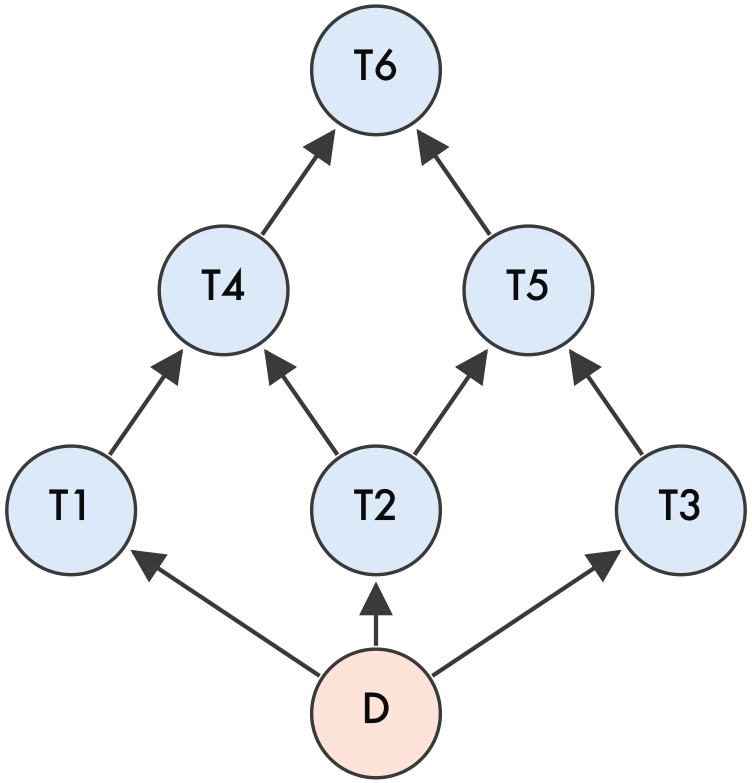
https://docs.dask.org/en/latest/graphviz.html

Key parallelism paradigms in data systems



L4 and L5: the left and middle columns; L6: the right column

Q3 · Task parallelism



Basic idea: split *tasks* across workers; if they read a common dataset, copy it to each (**replication**).

Given 3 workers:

1. Copy **D** to every worker
2. T1 → W1, T2 → W2, T3 → W3, in parallel
3. T1 done: T4 → W1 (T2 is done too). T2 done, but T5 also needs T3: W2 **idle**
4. T3 done: T5 → W2; W3 **idle**
5. T4, T5 done: T6 → W1; W2 **idle**

Your PA1 setup, except Dask places

Task parallelism: mechanics, pros, cons

- The scheduler **topologically sorts** the task graph: a task is eligible once its dependencies are done
- A "worker" can be a **core**, not just a node: thread-level parallelism instead of process-level
  - Dask: 4 worker nodes × 4 cores = **16 workers**

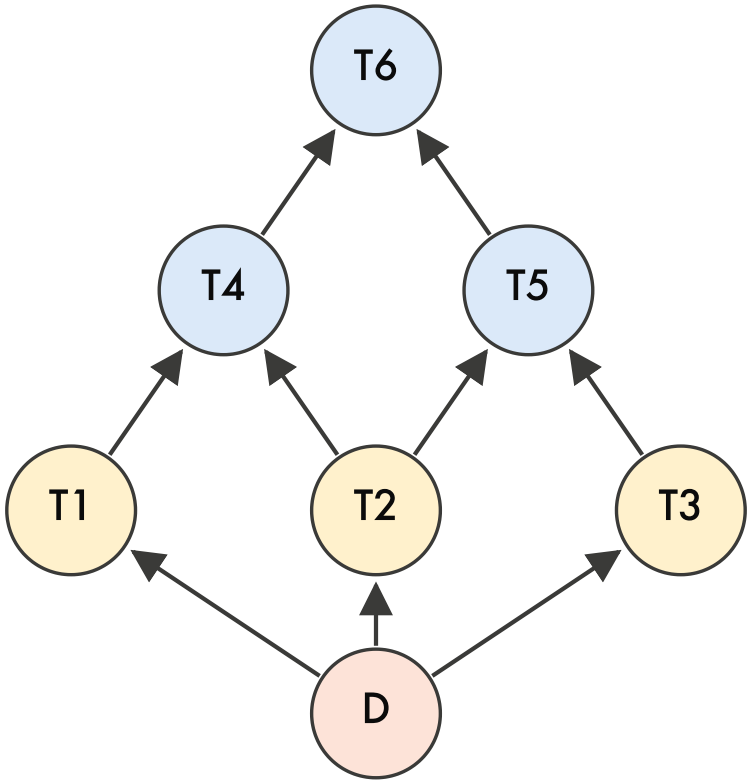
Pros

- Simple to understand, easy to implement
- Workers are **independent**: low software complexity

Cons

- Data **replication** across nodes wastes memory and storage
- **Idle** workers are possible

Degree of parallelism



Degree of parallelism: the largest amount of concurrency possible in the task graph, i.e. how many tasks can run at once.

- Here it is **3**: T1, T2, T3
- So more than 3 workers is **useless** for this workload
- And it **drops** over time: 2 at the next level, then 1

Q: How do we quantify the benefit of task parallelism?

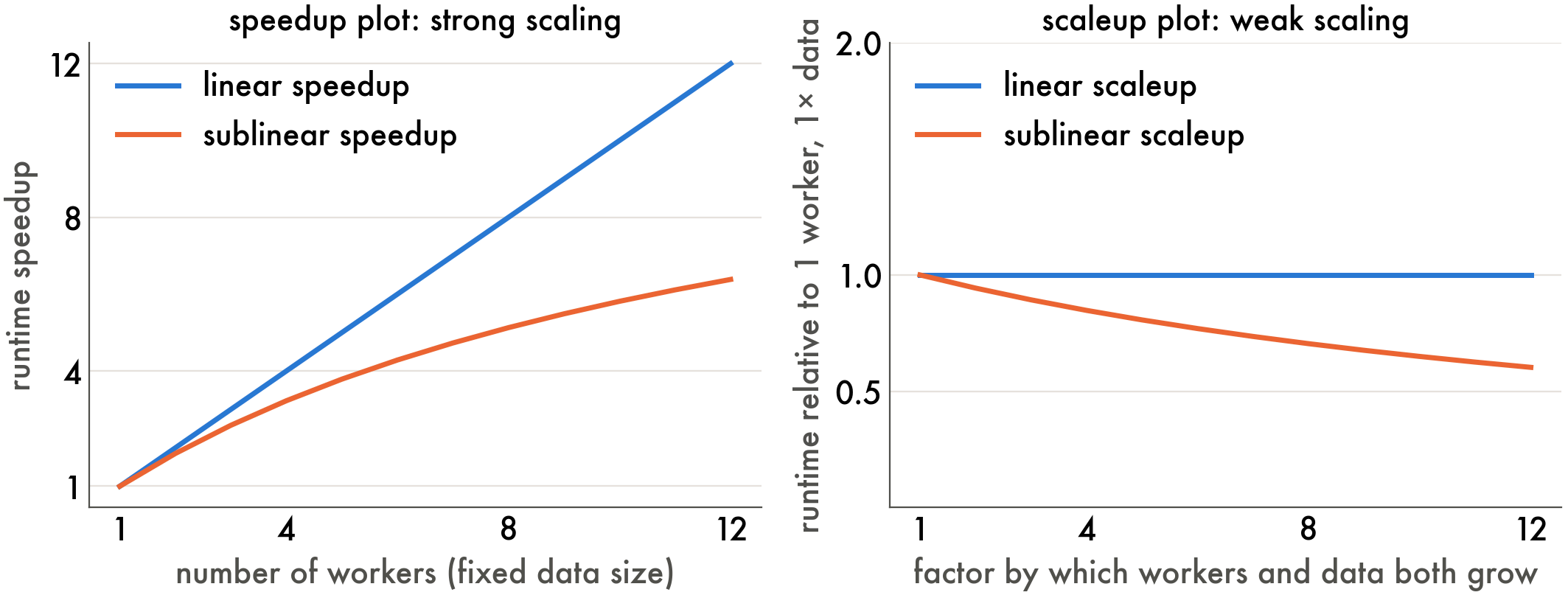
Quantifying the benefit: speedup

$$\text{Speedup} = \frac{\text{completion time with 1 worker}}{\text{completion time with } n \text{ workers}}$$

Q: Given n workers, can we get a speedup of n?

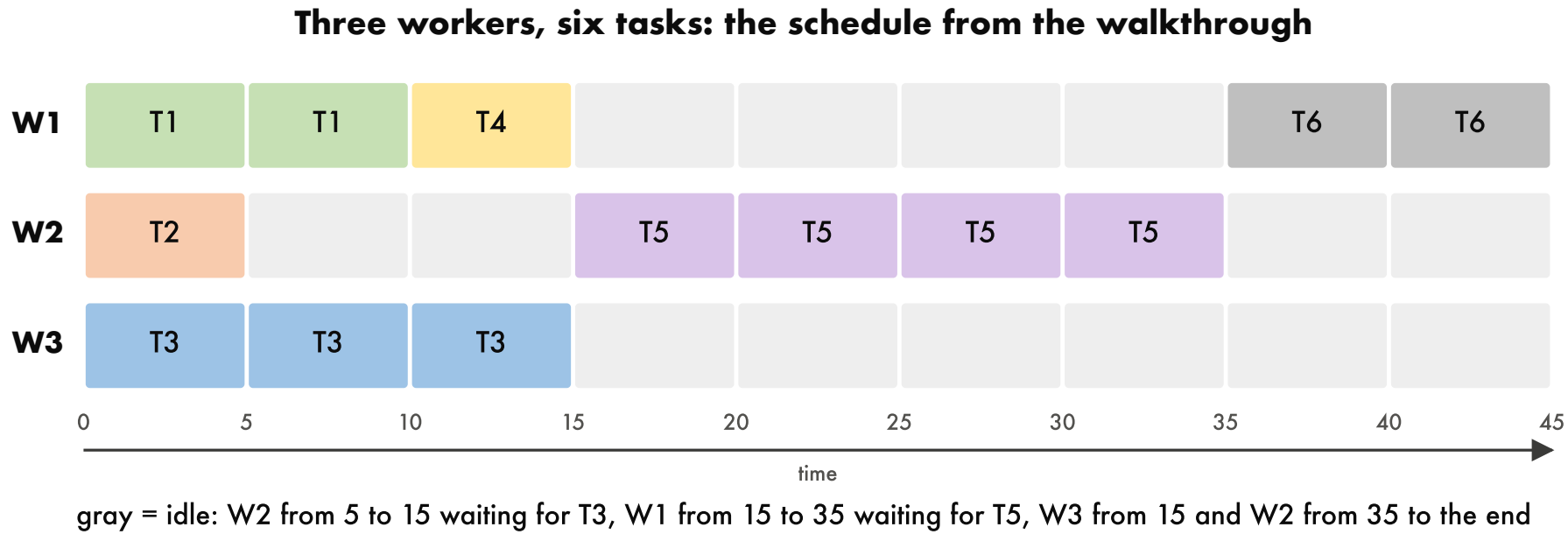
It depends: on the degree of parallelism, the structure of the task graph, the sizes of intermediate data, and the cost of moving them around.

Speedup versus scaleup



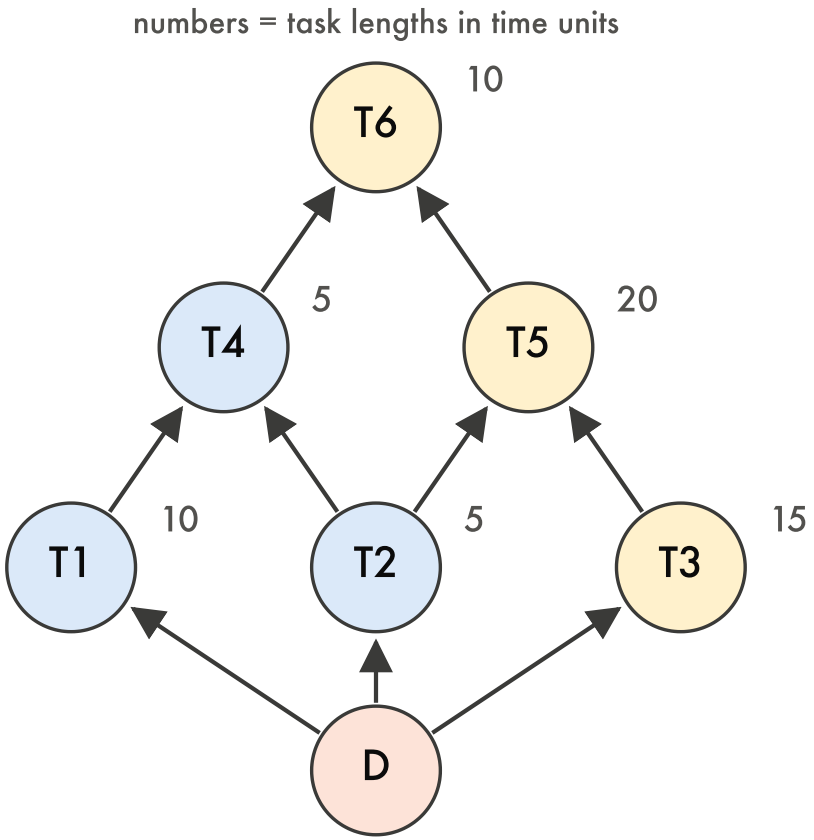
- **Speedup / strong scaling:** fix the data, add workers
- **Scaleup / weak scaling:** grow data and workers together; linear scaleup

Idle times in task parallelism



Task lengths: T1 = 10, T2 = 5, T3 = 15, T4 = 5, T5 = 20, T6 = 10. Varying task lengths, dependencies (T5 waits for T3), and a falling degree of parallelism leave workers **idle**.

The lower bound: the longest path



- A task cannot start before all of its predecessors finish, so the completion time on **any** number of workers is **lower-bounded by the longest path** in the task graph
- Here: D → T3 → T5 → T6 = 15 + 20 + 10 = **45**, which is exactly what our three-worker schedule achieved
- A fourth worker could not help; a scheduler that knows a worker will stay idle can **release** it, which saves money in the cloud

Exercise: two workers

Same task graph, same task lengths, but only **2 workers**.

1. Draw the Gantt chart. Which task goes where, and when?
2. What is the completion time, and the **speedup** over 1 worker?
3. What is the **degree of parallelism** now, and does the third worker ever pay for itself?

Q4 · Task parallelism in Dask

"Dask is a flexible library for parallel computing in Python."

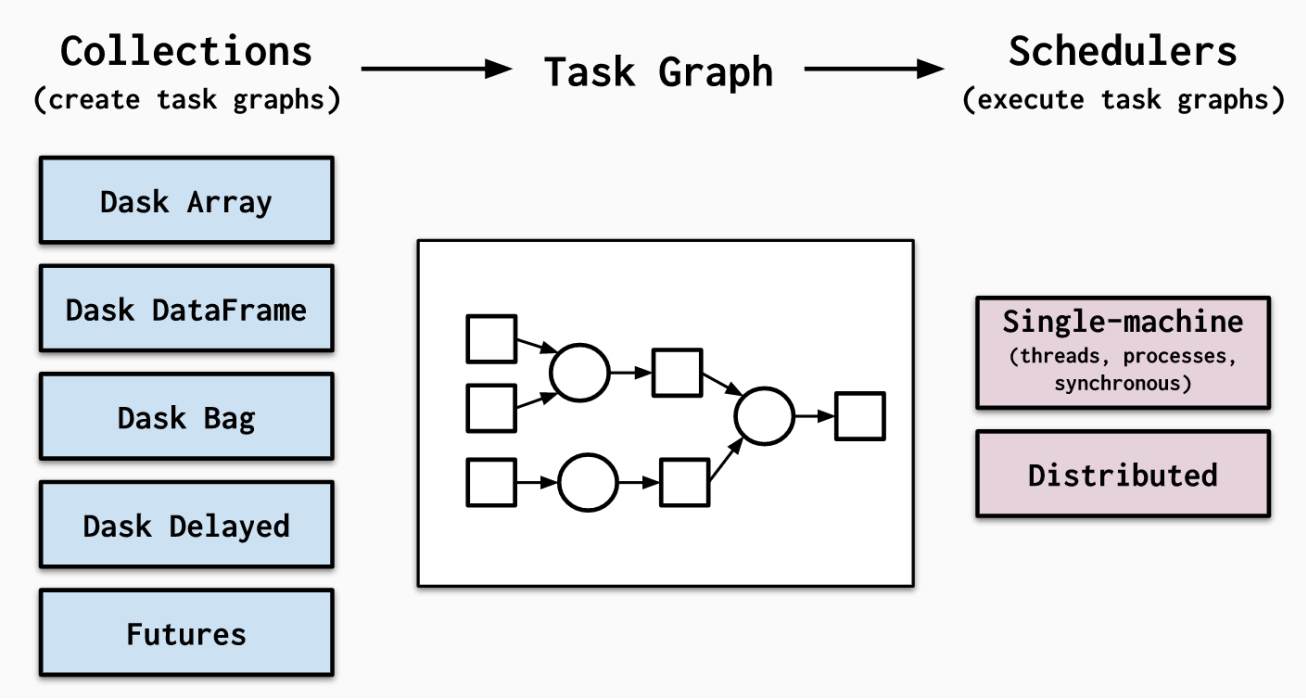
Two key components:

1. **APIs** for data science operations on large data
2. **Dynamic task scheduling** on multi-core and multi-node

Design desiderata: **Pythonic** (stay in the PyData stack, no JVM), **familiar** (the NumPy and pandas APIs), **scale up** (all cores) and **scale out** (a cluster), **flexible** (custom tasks), and **fast?** (optimized implementations under the APIs).

https://docs.dask.org/en/latest/

Dask's architecture



- **Collections** (array, DataFrame, bag, delayed, futures) create **task graphs**
- **Schedulers** execute them: single-machine (threads, processes, synchronous) or **distributed**
- The same graph can run on your laptop or on a cluster

https://docs.dask.org/en/latest/ · https://docs.dask.org/en/latest/scheduling.html

Dask's workflow: lazy evaluation

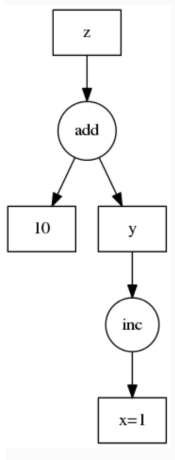
```
def inc(i):
    return i + 1

def add(a, b):
    return a + b

x = 1
y = inc(x)
z = add(y, 10)
```

User code, through Dask's API

```
d = {'x': 1,
     'y': (inc, 'x'),
     'z': (add, 'y', 10)}
```



- Operations on the data structures are **not executed immediately**; `compute()` triggers them
- Under the hood, a dataflow / task graph is built as a **dictionary**

Phases of computation

Rest of Dask's workflow for distributing a computation:

1. **Optimize** the task graph
2. **Serialize** the task graph (the dictionary)
3. **Communicate** it to the scheduler
4. The scheduler **divides** the work across cores and nodes
5. Each worker **does its job** and notifies the scheduler

Every phase is a place where time can go.

https://docs.dask.org/en/latest/phases-of-computation.html

Possible bottlenecks in Dask

Phase	What can go wrong
API implementation	Lack of operation coverage; **memory errors** in operations with poor scalability
Optimize	Poor or missed optimizations
Serialize and communicate	**Huge task graphs** take too long to serialize and ship
Schedule	Poor scalability of the scheduler itself
Execute	Machine and network **failures**

The task graph is both Dask's strength and its overhead.

Best practices 1: task graph size

Tuning the **number of tasks**:

- Avoid **too many** tasks: serialization, communication, and scheduling bottlenecks
- But avoid **too few** tasks: under-utilization of cores and nodes
- Be mindful of the available number of cores and nodes

Rough guidelines from the docs:

- Adjust the number of tasks by **fusing** operations ("batching") or **breaking them up**, and by tuning the **chunk size** (next slide)
- Use the **diagnostics dashboard** to watch task counts, core and node usage, completion

https://docs.dask.org/en/latest/best-practices.html · https://docs.dask.org/en/latest/diagnostics-distributed.html

Best practices 2: chunk size

Tuning the **data chunk size**:

- Avoid **too few** chunks (low degree of parallelism) but also **too many** (task graph overhead)
- Be mindful of the available DRAM

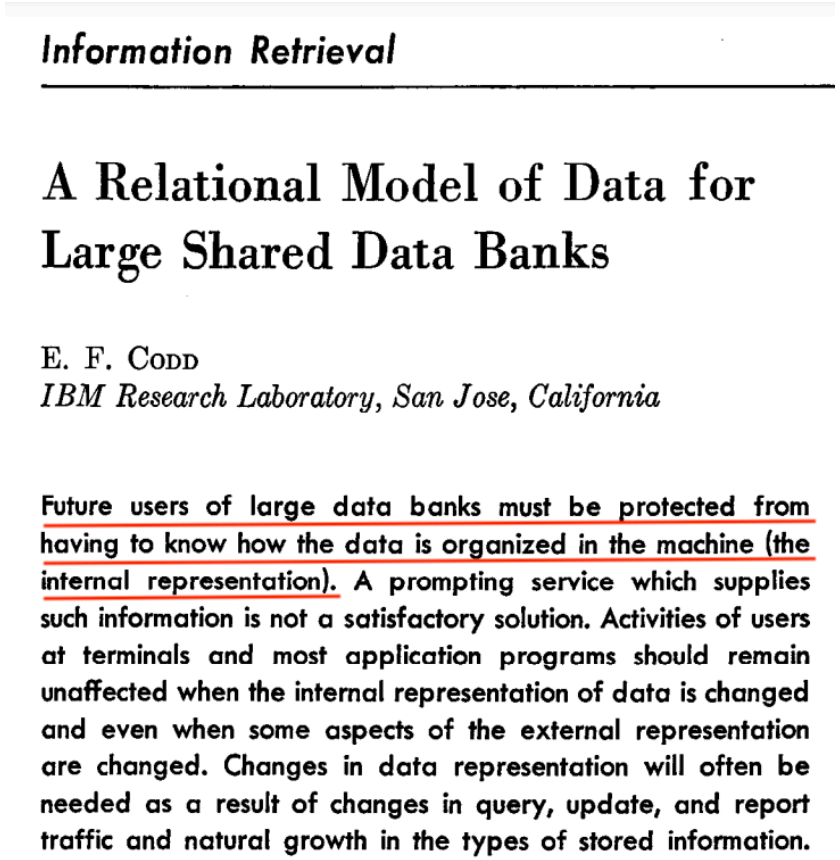
Rough guidelines from the docs:

- Number of chunks ≈ **3× to 10×** the number of cores, but
- cores × chunk size must be **< machine DRAM**, but
- chunk size should not be too small either (~**1 GB** is fine)

Q: Do you tune any of these when using an RDBMS?

https://docs.dask.org/en/latest/best-practices.html

Dask still lacks physical data independence



Information Retrieval

A Relational Model of Data for Large Shared Data Banks

E. F. Codd
IBM Research Laboratory, San Jose, California

Future users of large data banks must be protected from having to know how the data is organized in the machine (the internal representation). A prompting service which supplies such information is not a satisfactory solution. Activities of users at terminals and most application programs should remain unaffected when the internal representation of data is changed and even when some aspects of the external representation are changed. Changes in data representation will often be needed as a result of changes in query, update, and report traffic and natural growth in the types of stored information.

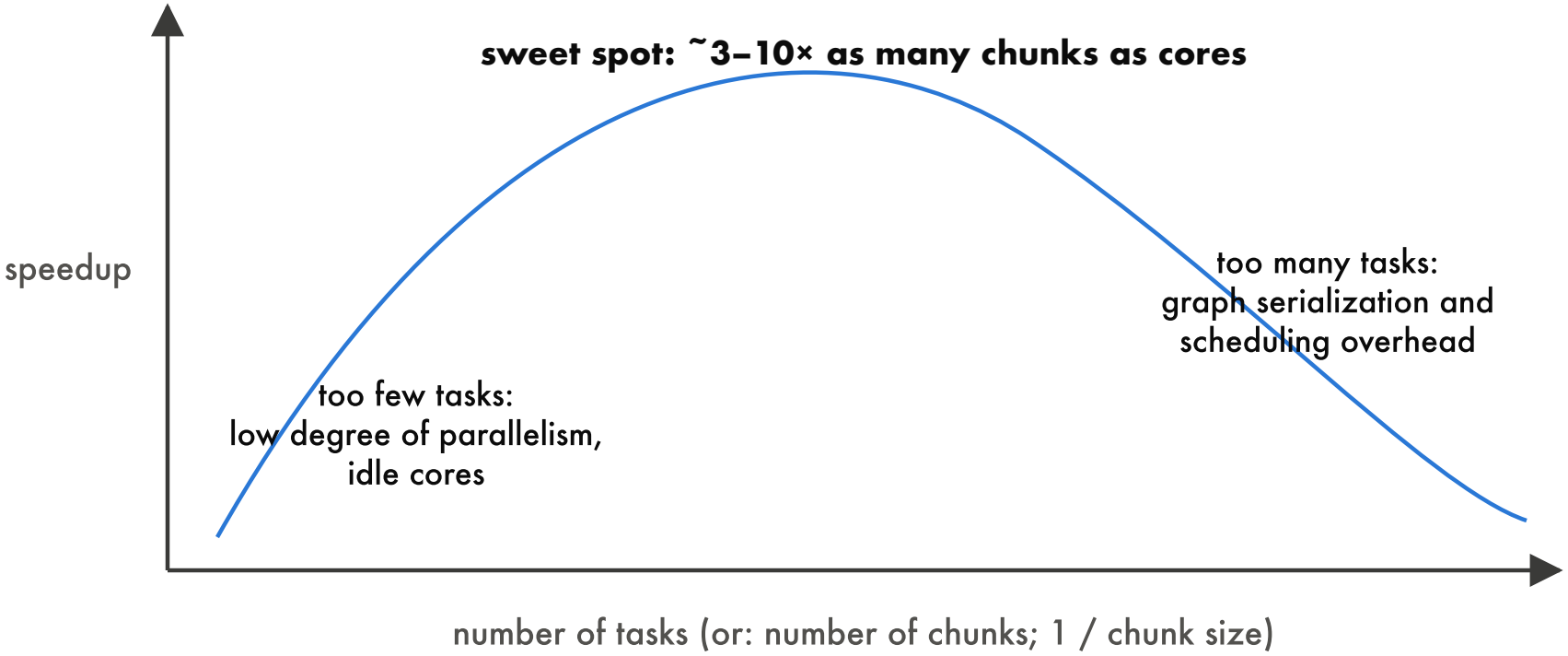
Codd, 1970: *"users of large data banks must be protected from having to know how the data is organized in the machine."*

- Half a century later, Dask users tune chunk sizes by hand
- The relational model separated the **logical** level from the **physical** level; most PyData tools do not

You analyze tabular data, you select, project, join, and group by: you are a database programmer.

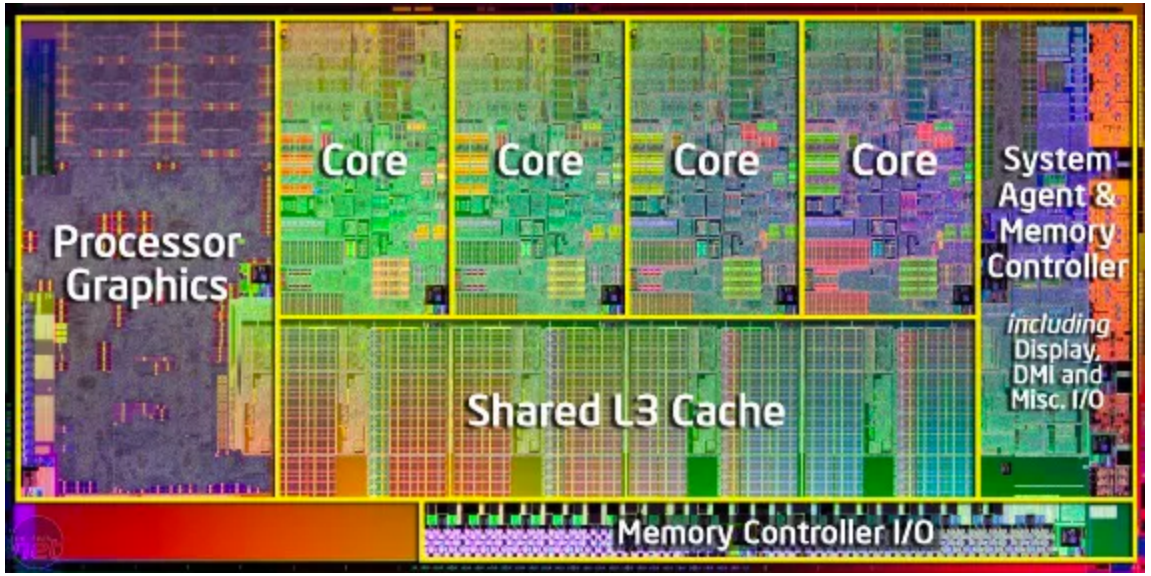
Codd, A Relational Model of Data for Large Shared Data Banks, CACM 1970. Meme: datasystemsfun.tumblr.com

Execution optimization tradeoffs



Be judicious in batching versus breaking up tasks, and in tuning chunk sizes.

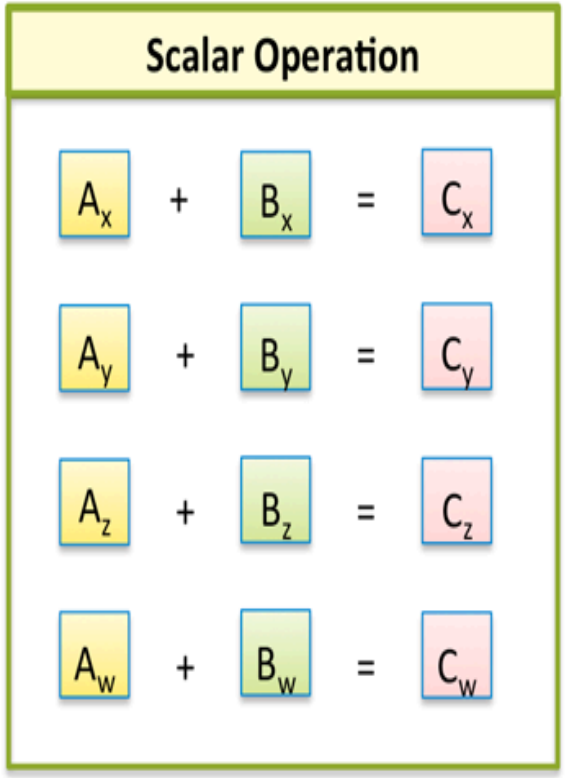
Q5 · Multi-core CPUs

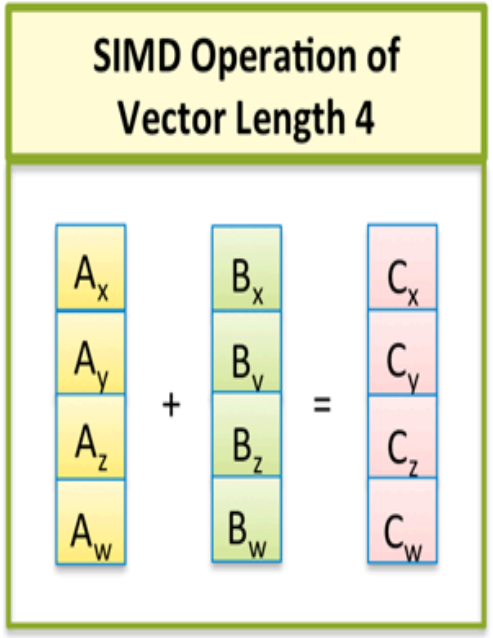


- Modern computers have multiple processors and multiple **cores** per processor, with a hierarchy of **shared caches** (recall L3)
- The **OS scheduler** controls which cores run which processes and threads, when
- Inside one node, parallelism comes in two more flavors: **SIMD** and **accelerators**

Die photo: Intel Core i7 (Sandy Bridge), Intel press materials.

Single instruction, multiple data



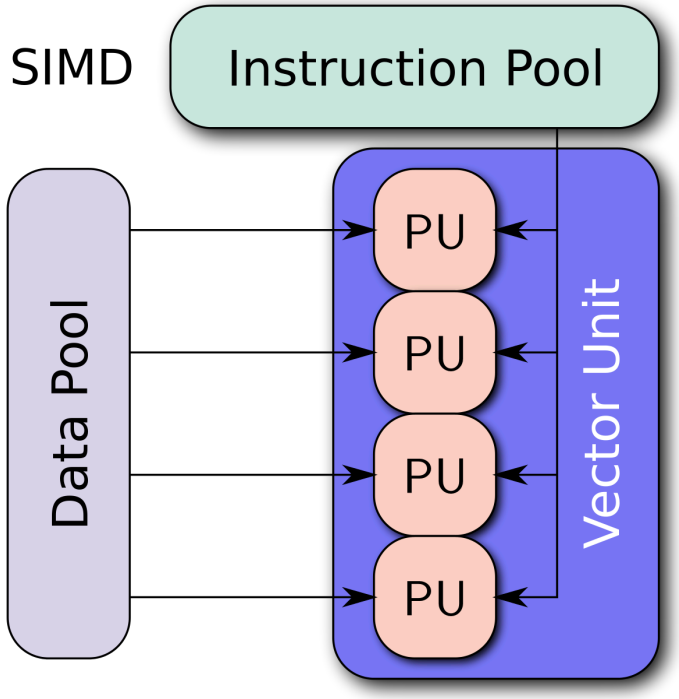


Intel® Architecture currently has SIMD operations of vector length 4, 8, 16

SIMD: different chunks of data are processed by the *same* instruction, shared by multiple processing units (PUs).

- Also called **vectorized** instruction processing, versus **scalar**
- One instruction adds 4, 8, or 16 numbers at once (Intel AVX)
- Data science workloads are **very** amenable to SIMD: NumPy, pandas, and every DL framework lean on it

SIMD generalizations

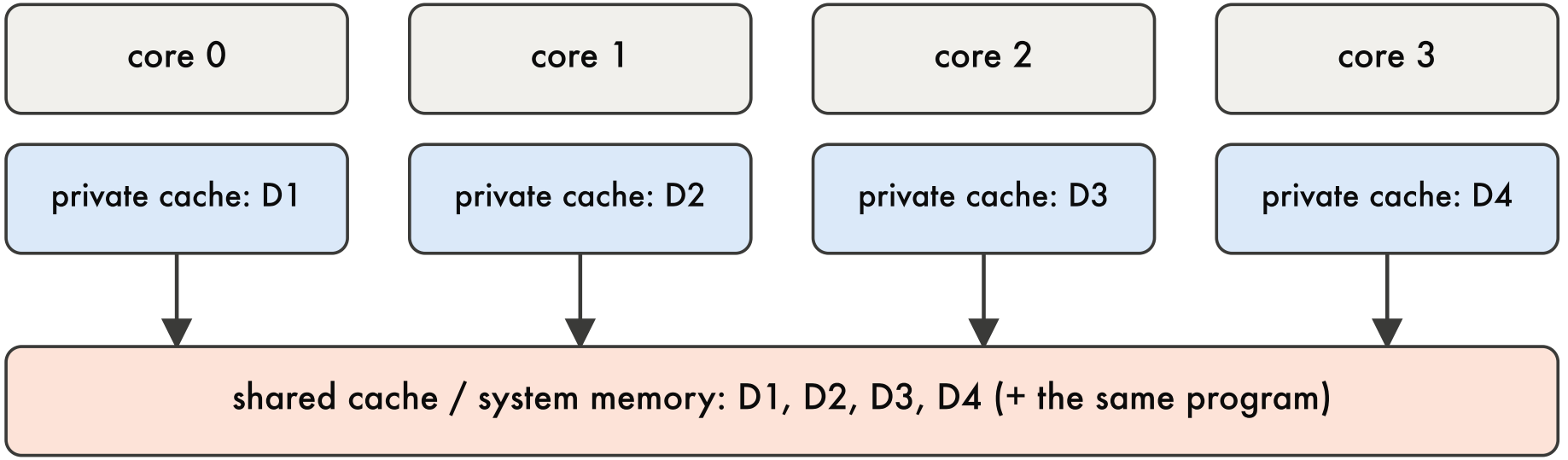


SIMT, single instruction multiple thread: different threads concurrently do SIMD; each thread gets a core or a whole PU. This is how GPUs work.

SPMD, single program multiple data: the same program over chunks of data, with processes or threads underneath, one chunk per core; works on any CPU, not just vector units.

SPMD is the most common form of parallel data processing at scale.

Data-parallel multi-core execution



SPMD: the same program on every core, each on its own chunk; results merged at the end

Quantifying efficiency: speedup, again

$$\text{Speedup} = \frac{\text{completion time with 1 core}}{\text{completion time with } n \text{ cores}}$$

- In data science computations, a useful surrogate for completion time is **instruction throughput**: **FLOPS**, floating-point operations per second
- Modern data processing programs, especially deep learning, run **billions of FLOPs** (GFLOPs) and up

Q: Given n cores, can we get a speedup of n? It depends, just as with task parallelism.

Amdahl's law

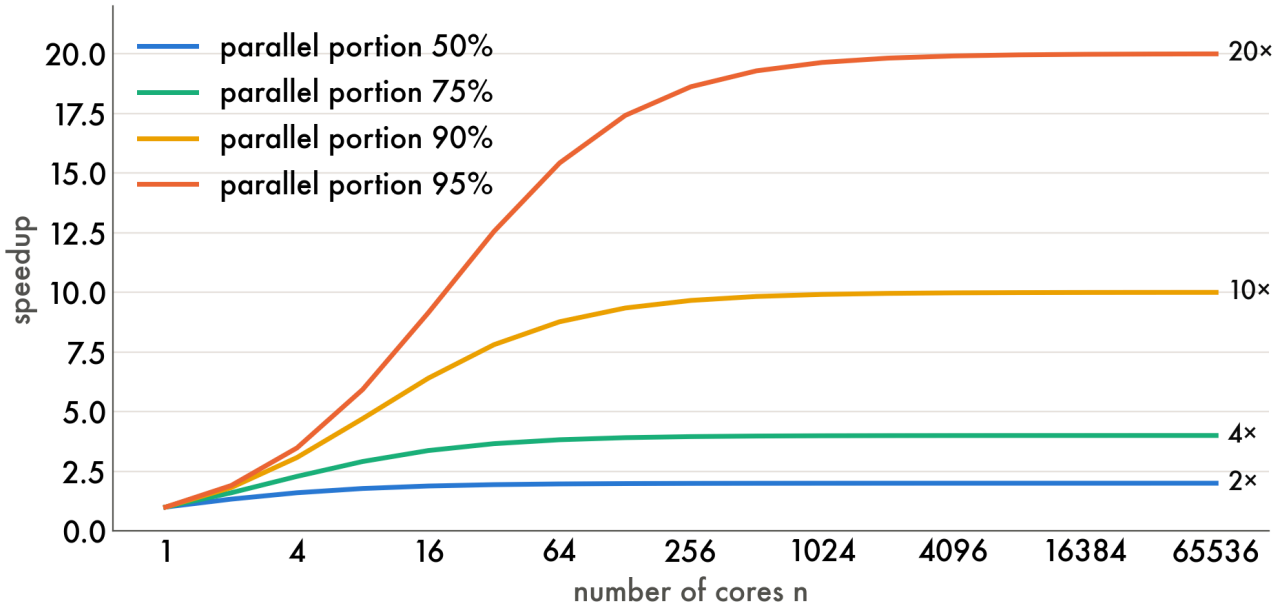
A program has two parts: one that **benefits** from multi-core parallelism (time T_{yes}) and one that **does not** (T_{no}: control, memory stalls, serial setup).

	1 core	**n cores**
parallel part	T_{yes}	T_{yes}/n
serial part	T_{no}	T_{no}

$$\text{Speedup} = \frac{T_{yes} + T_{no}}{T_{yes}/n + T_{no}} = \frac{n(1+f)}{n+f}, \qquad f = \frac{T_{yes}}{T_{no}}$$

Amdahl's law upper-bounds the possible speedup: the serial part never shrinks.

Amdahl's law, plotted

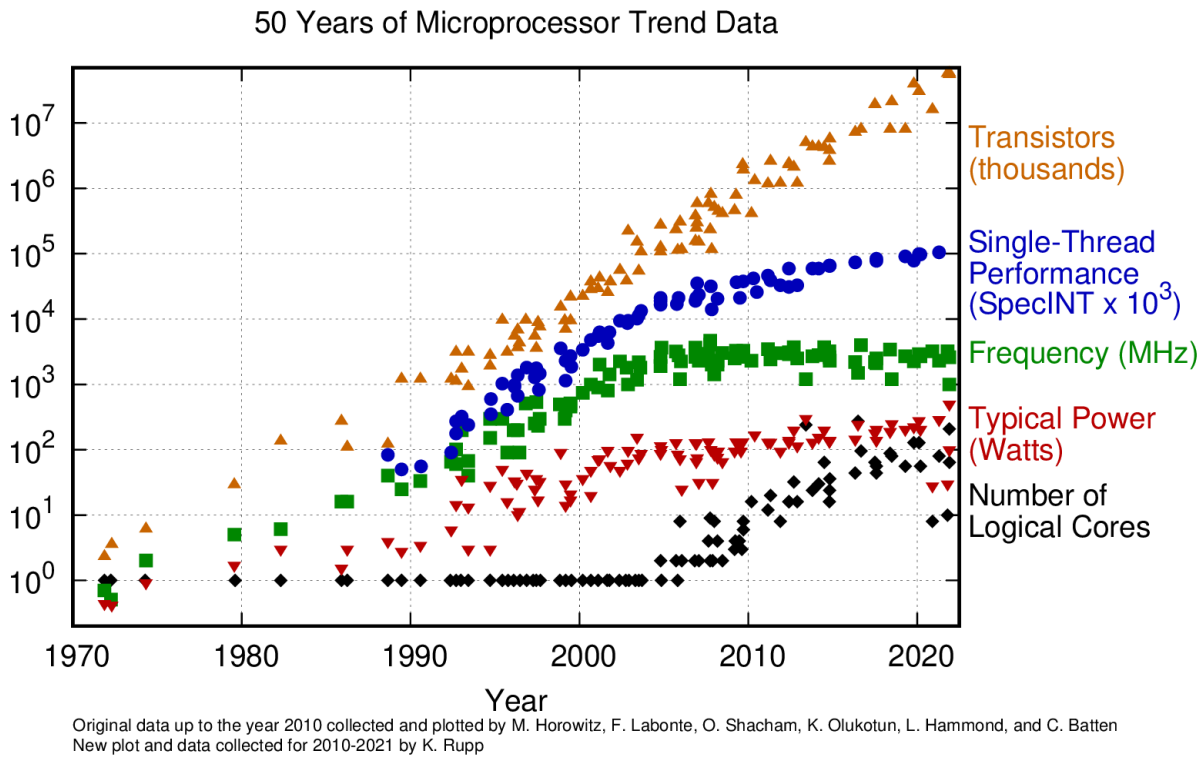


Parallel portion $= f/(1+f)$.

- At 50% parallel, speedup never passes **2×**
- At 95% parallel, 65,536 cores give **20×**
- The curve flattens where the serial part dominates

Make the serial part small before you buy more cores.

Hardware trends on parallelism

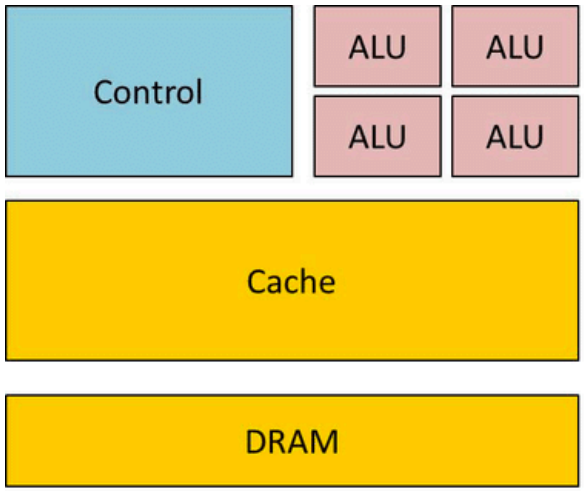


- Multi-core CPUs grew rapidly in the early 2000s, then hit physical limits: packing efficiency and **power**
- The end of **Moore's law** and of **Dennard scaling**: clock speed and power per transistor stopped improving

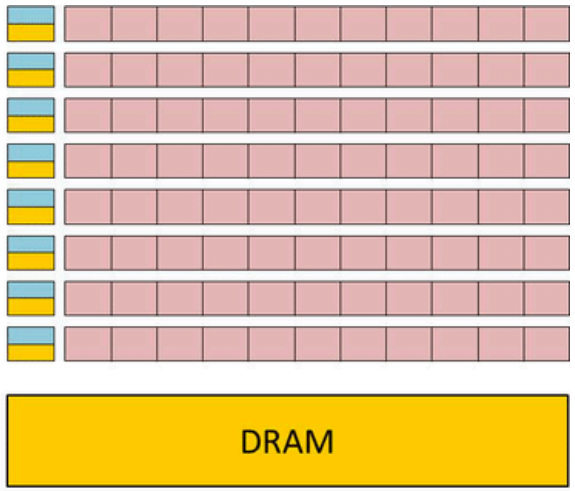
Takeaway: general-purpose CPUs cannot sustain FLOP-heavy programs like deep nets. That motivated **accelerators** for some classes of programs.

Karl Rupp, 42 Years of Microprocessor Trend Data.

Hardware accelerators: GPUs



CPU

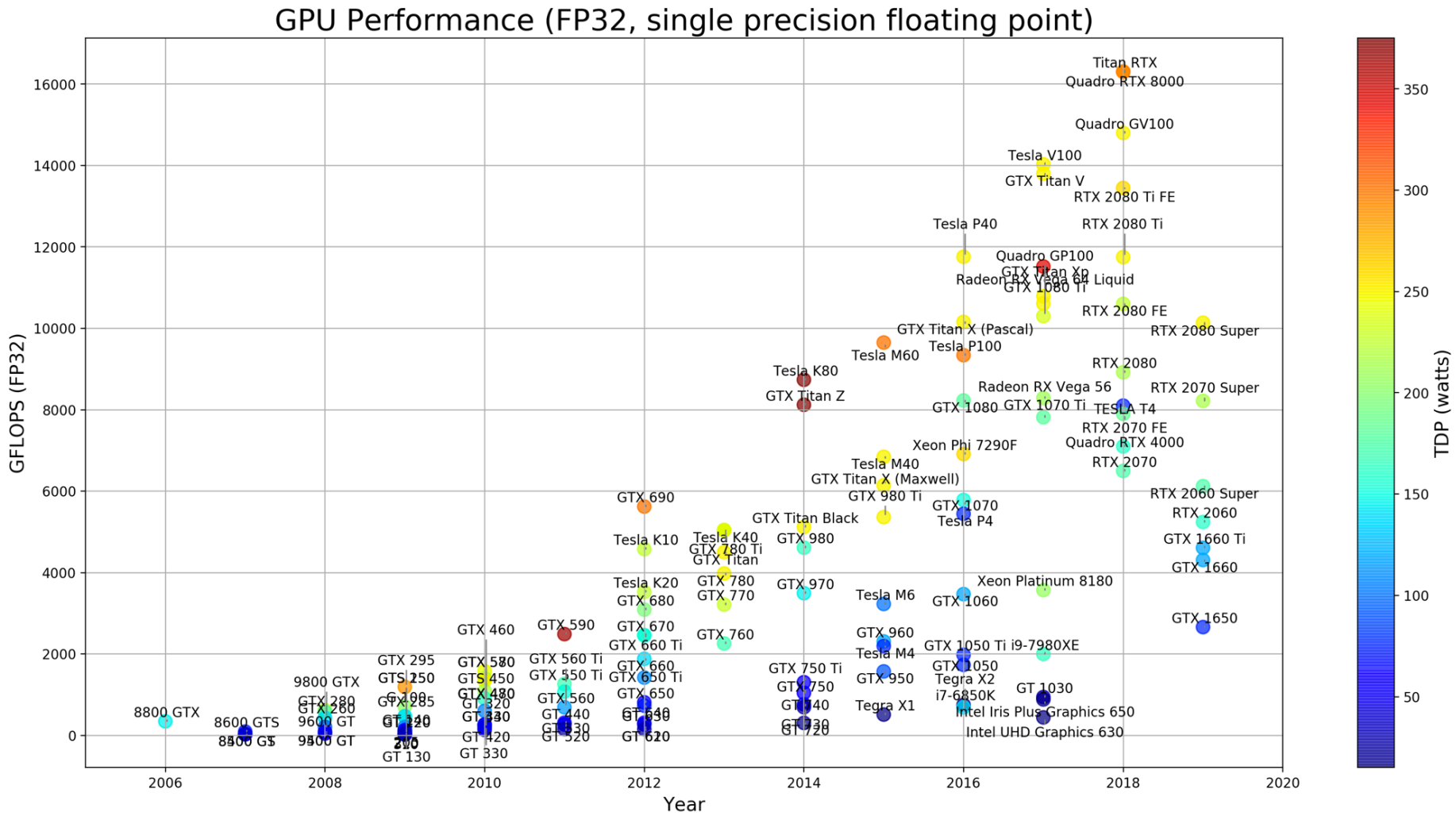


GPU

Graphics Processing Unit: tailored for matrix and tensor operations.

- Basic idea: **tons of ALUs**, massive data parallelism: SIMD on steroids
- Popularized by NVIDIA in the early 2000s for games and graphics; now ubiquitous in deep learning
- **CUDA** (2007), then wrapper libraries: cuDNN, cuSparse, cuDF (RAPIDS)
- An **A100** offers ~20 TFLOPS in FP32

GPUs on the market

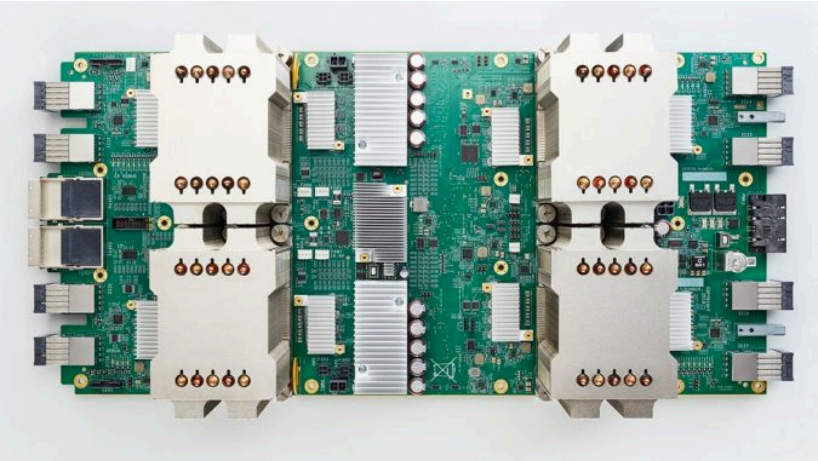


GPU FP32 performance over time; color: TDP in watts.

Other hardware accelerators



Tensor Processing Unit (TPU): an application-specific integrated circuit (**ASIC**) created by Google in the mid-2010s, used for AlphaGo; even more specialized tensor ops for DL inference, ~45 TFLOPS.



Field-Programmable Gate Array (FPGA): configurable for *any* class of programs; ~0.5 to 3 TFLOPS but very low power; cheaper; new hardware-software integrated stacks for ML.

Comparing modern parallel hardware

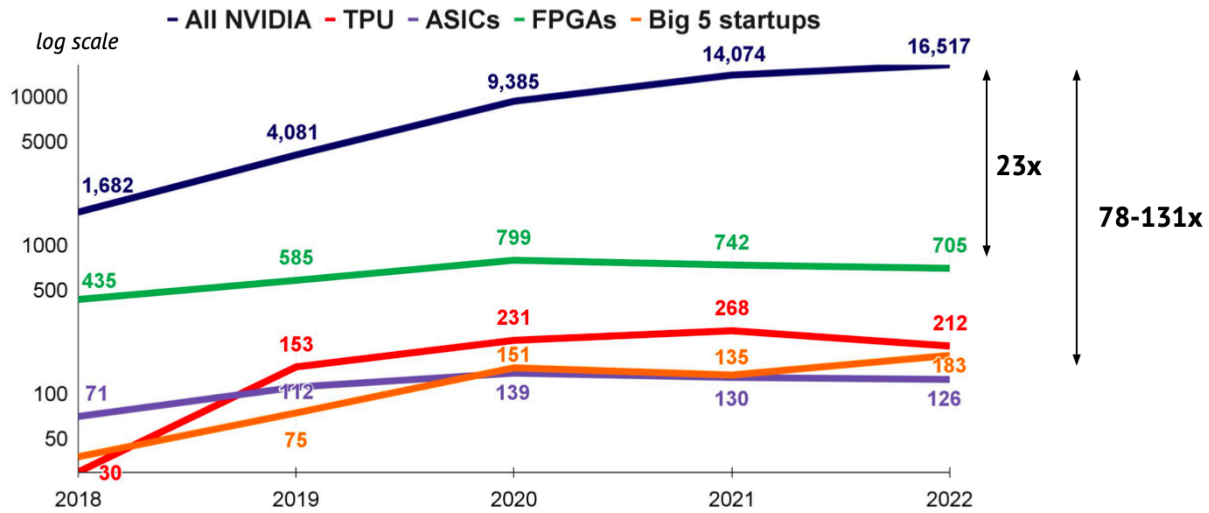
	Multi-core CPU	GPU	FPGA	ASIC (e.g. TPU)
Peak FLOPS	Moderate	High	High	Very high
Power consumption	High	Very high	Very low	Low to very low
Cost	Low	High	Very high	Highest
Generality / flexibility	Highest	Medium	Very high	Lowest
Fit for DL training	Poor	**Best**	Poor	Potential, not mass market
Fit for DL inference	Moderate	Moderate	Good	**Best**
Cloud vendor support	All	All	All	GCP

After https://www.embedded.com/leveraging-fpgas-for-deep-learning/

NVIDIA's hold remains strong

NVIDIA's chips are the most popular in AI research papers...and by a massive margin

- **GPUs are 131x more commonly used than ASICs, 90x more than chips from Graphcore, Habana, Cerebras, SambaNova and Cambricon combined, 78x more than Google's TPU, and 23x more than FPGAs.**



Zeta Alpha

stateof.ai 2022

State of AI Report 2022, stateof.ai (data: Zeta Alpha).

Review questions

1. Briefly explain three benefits of large-scale data in data science.
2. What is a dataflow graph? Give an example from a data system. How does a task graph differ?
3. Explain one pro and one con of task parallelism.
4. Name one scalability bottleneck that Dask still faces.
5. What is the degree of parallelism of a task graph?
6. What is speedup? How is it different from scaleup?
7. Is linear speedup always possible with task parallelism?
8. What is SIMD? Why is vectorized data processing critical?
9. What is the point of Amdahl's law?
10. Give one pro and one con of a TPU versus a GPU.

Key takeaways

1. **Large-scale data** pays off: granular phenomena, new applications, lower variance
2. A **dataflow graph** or **task graph** describes a parallel computation at the logical level
3. **Task parallelism** is simple and robust, but replicates data and leaves workers idle
4. **Speedup** is bounded by the longest path in the task graph, and by **Amdahl's law** inside a node
5. **Dask** turns PyData code into task graphs, and the graph size is its main tuning knob
6. Inside one node, **SIMD** and **GPUs** do the heavy lifting for data science